

Aza I MCP Integration Guide

Overview

This guide is the definitive technical reference for integrating with Aza using the Model Context Protocol (MCP). It covers MCP server design principles, the integration contract (request/response formats), streaming support, tool registration, authentication, and best practices.

For understanding which integration type is right for your use case and step-by-step onboarding instructions, refer to the Aza I Onboarding Guide [WIP].

What is MCP in Aza

The Model Context Protocol (MCP) is the recommended and standardized protocol for integrating external tools and agents with Aza. MCP establishes a dynamic protocol allowing Aza's RBO to discover, register, and invoke tools from remotely running servers through a well-defined request/response model with schema validation and standardized error handling.

MCP servers function as independent processes that expose a collection of specialized tools and resources. In Aza, MCP is used for both Plugin Actions (discrete data retrieval) and Plugin Agents (autonomous sub-agents with full response ownership).

MCP Server Design Recommendations

TASK-ORIENTED TOOL DESIGN

The recommended approach for MCP server implementation is the Task-Oriented Tool Integration model. Your server should:

- Expose a limited set of carefully designed, task-oriented tools (not raw API endpoints)
- Organize complex workflows into purpose-built multi-step processes
- Optimize all tool descriptions specifically for LLM consumption

Focus on User Goals, Not API Structure

- Design tools around what users want to accomplish
- Cover a general scope as the tool's responsibility instead of modeling tools at the query level
- Abstract implementation complexity, a single tool may orchestrate multiple API calls behind the scenes

Task-Oriented (Recommended)	API-Oriented (Avoid)
complete_customer_order	update_database_record
get_team_information handles employee, manager, and team lookups internally	get_employee_by_id, get_manager_by_id, get_team_by_manager_id
One tool covering a category of related queries	One tool per query type (3 tools for 3 specific queries)

Create Purpose-Built Workflows for Complex Tasks

For complex operations, break them into logical steps with validation points:

1. `prepare_database_migration` → validates and stages changes
2. `review_database_migration` → shows what will happen
3. `complete_database_migration` → applies changes

WRITE LLM-OPTIMIZED TOOL DESCRIPTIONS

The tool description is how the RBO determines whether to invoke your tool. Poorly written descriptions lead to incorrect routing either missed invocations or false triggers.

Guidelines:

- Be explicit and detailed about what each tool does
- Include clear examples of inputs and outputs
- Use structured formats (XML tags recommended for example queries)
- Explicitly state when to use and when NOT to use each tool
- Keep descriptions within 500 tokens all tool descriptions roll up to the RBO's top-level prompt which has a 200K token limit. If your description exceeds 500 tokens, your tool modeling likely needs refinement.

LLM-optimized tool description should have the below information

```
Search AWS documentation using the official AWS Documentation Search API.
## Usage
This tool searches across all AWS documentation for pages matching your search phrase.
Use it to find relevant documentation when you don't have a specific URL.

## When to Use
- User asks about AWS service documentation
- User needs to find a specific AWS guide or reference

## When NOT to Use
- User asks about internal Amazon policies (use MyHR instead)
- User asks about non-AWS technical documentation

## Search Tips
- Use specific technical terms rather than general phrases
- Include service names to narrow results (e.g., "S3 bucket versioning" instead of jus

<example_queries>
Can you find documentation about Lambda function URLs?
How do I configure S3 bucket versioning?
</example_queries>
```

UNDERSTANDING RBO ORCHESTRATION CAPABILITIES

When designing your tool's scope and responsibility, it is critical to understand what the RBO can and cannot do:

1. **Capability Description** - Intent-Based Routing RBO routes queries based on intent matched against your tool description. Your description defines the scope of queries your tool handles.
2. **Sequential Invocation** - RBO can invoke Tool A, use its output, then invoke Tool B when one tool's result is needed as input for another.
3. **Parallel Invocation** - RBO can break conjunction queries (e.g., "Get me X and Y") into sub-queries and invoke multiple

tools in parallel, then combine responses without adding or removing information.

4. **Context Sharing** - During a conversation, any tool can receive previous conversation history to provide contextual responses. [Context Sharing SOP]

Important: The RBO does not provide any mechanism to explicitly configure parallel or sequential tool calling, or any custom prompting to orchestrate across MCP tools for specific queries. Invocation strategy is entirely LLM-determined based on the configured prompt and scope descriptions.

WHY APIS SHOULD NOT BE DIRECTLY EXPOSED AS MCP TOOLS

While auto-generating an MCP server from existing API specifications might seem convenient, this approach leads to suboptimal results:

- **Tool Overload** - Most APIs have many parameters. LLMs perform poorly when presented with too many tools, becoming confused about which to select.
- **Context Efficiency & Latency** - LLMs perform significantly better with focused context. Larger prompts with extensive tool lists increase inference latency.
- **Documentation Mismatch** - API docs are written for human developers. LLMs require more explicit guidance, examples, and clear instructions.
- **Low-Level Operations** - APIs are designed for resource management. LLMs work best with task-oriented tools aligned to user goals.
- **Missed Opportunities** - Auto-generation misses opportunities for creating purpose-built tools that better serve user needs.

If you have existing APIs, use this approach:

1. Start with auto-generation as a baseline only
2. Aggressively prune aim for 10–20 tools maximum
3. Rewrite all descriptions optimize for LLM consumption, don't keep auto-generated descriptions
4. Add purpose-built tools for complex workflows
5. Test thoroughly with real-world query scenarios

WELL-DESIGNED TOOL CHECKLIST

- **Clear Purpose** - Each tool has a distinct, well-defined purpose aligned with user goals
- **Optimized Description** - Description is explicit, detailed, includes examples, and fits within 500 tokens
- **Appropriate Abstraction** - Tool hides unnecessary complexity; may orchestrate multiple API calls internally
- **Explicit Boundaries** - Description states what the tool does AND does not handle
- **Consistent Routing** - When tested, the RBO consistently selects your tool for relevant queries and avoids it for irrelevant ones
- **User-Centric Design** - Tool is designed around user goals, not system architecture

Aza MCP Integration Structures

REQUEST STRUCTURE FOR TOOL INVOCATION | TOOLS/CALL

Request Headers

The following fields are included in the headers when the request is signed by Aza:

```
{
  "x-employee-id": "123456789", // Employee id of the user Use Transitive auth t
  "x-person-id": "5b3d5363-5f0e-437d-8e2d-107f0cf19c05", // Person id of the user
  "x-alias": "abcde" // Alias of the user Use Transitive auth token for AtoZ Aza
  "x-amzn-transitive-authentication-token": "JSHASDSAJYTRTYUYG" // Transitive auth
  "X-Amzn-Bedrock-AgentCore-Runtime-Custom-transitive-authentication-token": "JSHAS
}
```

Note - For AgentCore integration type all headers will be prefixed with X-Amzn-Bedrock-AgentCore-Runtime-Custom- as per the AgentCore custom header guidelines.

Request Body

The request follows the standard MCP `tools/call` specification:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "method": "tools/call", //standard function call in MCP
  "params": {
    "name": "search_documentation", // tool name
    "arguments": { //tool specific input request body
      "search_phrase": "lambda function", //tool specific input
      "limit": 2 //tool specific input
    }
    "_meta": {
      "additionalProperties": {
        "conversationHistory": "https:...", // This will send a presigned s3 v
        "attachments": [] // This will send a presigned s3 with attachment lir
        "toolConversationId": "" // If tool sends a conversationId via metadat
        "userMessage": "string" // Actual user message
      }
    }
  }
}
```

CONVERSATION HISTORY FORMAT

When `isConversationHistoryRequired` is enabled, the conversation history is provided as a pre-signed S3 URL. The content at that URL follows this structure:

```
{
  "messages": [{
    "actionExecution": null,
    "actionReview": null,
    "attachments": null,
    "body": "Based on your current time off balances, you have:\n- 12.0 days of IN
    "enhancedMessage": {
      "actionItems": [{
```

```

        "link": {
            "description": null,
            "icon": null,
            "title": "Time off history",
            "url": "https://atoz.amazon.work/time/history"
        }
    }, {
        "link": {
            "description": null,
            "icon": null,
            "title": "Request Time off",
            "url": "https://atoz.amazon.work/time"
        }
    }
    ],
    "formFooterText": null,
    "formSubmitButtonText": null,
    "textFormatSegments": []
},
"messageId": "3fef440b-1341-499c-8f5b-41aeeae9ef3",
"mfeAttributes": [],
"plugins": ["MYHR"],
"sourceAttribution": [{
    "botId": null,
    "citationNumber": 1,
    "snippet": "",
    "textMessageSegments": null,
    "title": "SOP: Absence Request Process - ESP Operations",
    "updatedAt": null,
    "url": "https://inside.hr.amazon.dev/content/inside/in/en/employment/time-
}, {
    "botId": null,
    "citationNumber": 2,
    "snippet": "",
    "textMessageSegments": null,
    "title": "Personal Time Away (Unpaid) FAQ - Global",
    "updatedAt": null,
    "url": "https://inside.hr.amazon.dev/content/inside/in/en/employment/polic
}, {
    "botId": null,
    "citationNumber": 3,
    "snippet": "",
    "textMessageSegments": null,
    "title": "MyTime - Global",
    "updatedAt": null,
    "url": "https://inside.hr.amazon.dev/content/inside/in/en/employment/time-
}, {
    "botId": null,
    "citationNumber": 4,
    "snippet": "",
    "textMessageSegments": null,
    "title": "Centralized Timekeeping (CTK) - IND\r\n",
    "updatedAt": null,
    "url": "https://inside.hr.amazon.dev/content/inside/in/en/employment/time-
}],
"time": 1.755144888E9,

```

```

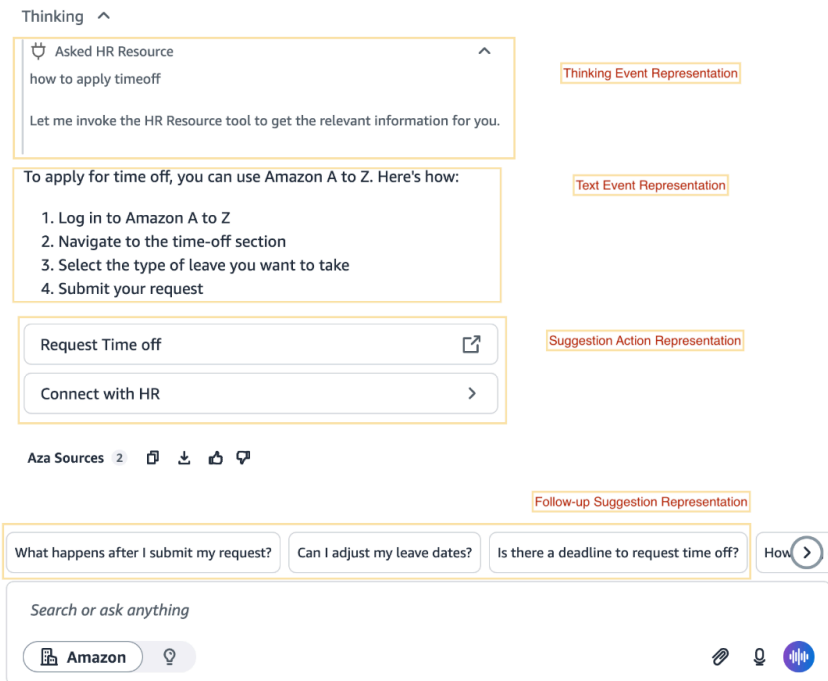
      "type": "SYSTEM"
    }, {
      "actionExecution": null,
      "actionReview": null,
      "attachments": [],
      "body": "what is my time off baalnce ",
      "enhancedMessage": null,
      "messageId": "5399eb05-08bf-4b6a-a4f7-a7f0c316d8f2",
      "mfeAttributes": null,
      "plugins": null,
      "sourceAttribution": null,
      "time": 1.755144887E9,
      "type": "USER"
    }],
    "nextToken": null
  }

```

Attribute	Description
body	The text content of the message
type	"USER" for user messages, "SYSTEM" for Aza/plugin responses
time	Epoch timestamp of the message
messageId	Unique identifier for the message
enhancedMessage	Rich content (links, suggestions, actions) attached to the response
sourceAttribution	Citations/sources referenced in the response
plugins	Which plugin(s) generated this response
mfeAttributes	MFE components rendered in this response
attachments	Files uploaded by the user

RESPONSE STRUCTURE

Response Event Representations



Basic Response Structure

```
{
  "jsonrpc": "2.0",
  "id": "req-123",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Response in LLM understandable format"
      },
      {
        "type": "image",
        "data": "<pre-signed s3 url of the image>"
      },
      {
        "type": "resource_link",
        "name": "<name of the file>",
        "uri": "<pre-signed s3 url of the file>",
        "mimetype": "<mimetype eg:application/pdf, application/msword, text/plain>"
      }
    ],
    "isError": false
  }
}
```

CONTENT TYPES

The `content` array can contain one or more of the following types:

Content Type	Schema Type	Description	How Aza Processes It
Text	McpSchema.TextContent	Plain text or stringified JSON (MetadataEvent/FollowUpSuggestionsEvent)	Extracted and returned as system response to UI/RBO. If isLLMProcessingRequired: true (default), stringified JSON is sent to the RBO for LLM interpretation.
Image	McpSchema.ImageContent	Pre-signed S3 URL of an image	Rendered directly in the UI
Resource Link	McpSchema.ResourceLink	Pre-signed S3 URL of a document file	Content extracted from S3, sent to LLM for summarization. Supports: PDF, DOC, DOCX, XLS, XLSX, CSV, TXT, MD. Unrecognized formats default to TXT.

Aza supports two types of response mechanism

Sync

Sync mechanism is required for all Plugin Actions as Aza requires full plugin response to understand the further context.

```
{
  "jsonrpc": "2.0",
  "id": "req-123",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Response in LLM understandable format"
      }
    ],
    "isError": false
  }
}
```

Async(Streaming)

For Plugin Agents that need to stream responses in real time, Aza supports streaming via JSON-RPC Notifications sent as Server-Sent Events (SSE). Streaming reduces user-perceived latency by delivering content as it becomes available rather than waiting for the complete response.

Streaming consists of three event types, all sent as notifications before the final `CallToolResult`:

TEXTOUTPUTEVENT | STREAMING TEXT

Used for streaming text response chunks to the user in real time.

```
{
  "jsonrpc": "2.0",
  "id": "req-123",
  "method": "notifications/message",
  "params": {
    "TextOutputEvent": "{\"conversationId\":\"1db47a7c-9b2f-47d0-aec6-5cd794cde046\"}"
  }
}
```

LLMREASONINGEVENT | STREAMING THINKING

Used for streaming the agent's reasoning/thinking trace separately from the response. This is rendered in the UI as a collapsible "Thinking" section and helps reduce user-perceived first token latency users see the agent is working before the actual response starts.

```
{
  "jsonrpc": "2.0",
  "id": "req-123",
  "method": "notifications/message",
  "params": {
    "LLMReasoningEvent": "{\"reasoningType\":\"THINKING\", \"llmReason\":\"Analyzir"
  }
}
```

FINAL RESPONSE (CALLTOOLRESULT)

After streaming is complete, the tool must send a final `CallToolResult` containing:

- `MetadataEvent` (mandatory) The complete response with all metadata
- `FollowUpSuggestionsEvent` (optional) Suggestion chips

```
{
  "jsonrpc": "2.0",
  "id": "req-453",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"metadataEvent\": {}}"
      },
      {
        "type": "text",
        "text": "{\"followUpSuggestionsEvent\": {}}"
      }
    ],
    "isError": false
  }
}
```

STREAMING SEQUENCE

A typical streaming interaction follows this sequence:

- | | |
|------------------------|--|
| 1. [LLMReasoningEvent] | → "Analyzing user's query about leave policy..." |
| 2. [LLMReasoningEvent] | → "Checking leave balance for the user..." |
| 3. [TextOutputEvent] | → "Based on the India Leave Policy, " |
| 4. [TextOutputEvent] | → "here's how you can request sick leave:\n\n" |
| 5. [TextOutputEvent] | → "For planned sick leave:\n- If less than..." |
| 6. [CallToolResult] | → Final MetadataEvent + FollowUpSuggestionsEvent |

Note: Thinking rendering via `LLMReasoningEvent` is supported with MCP streaming only. It is not available with the legacy Bot Client integration.

OTHER EVENTS

For Plugin Agents that want full control over the response (typically with `isLlmProcessingRequired: false`), the response should include a `MetadataEvent` and optionally a `FollowUpSuggestionsEvent`, both sent as stringified JSON within `TextContent`.

MetadataEvent

The `MetadataEvent` is the primary structure for sending enhanced response.

```
{
  "jsonrpc": "2.0",
  "id": "req-453",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"metadataEvent\":{\"conversationId\":\"1db47a7c-9b2f-47d0-a\""}"
      }
    ],
    "isError": false
  }
}
```

Field	Required	Description
conversationId	Optional	ConversationId for tracking in Aza DB and telemetry. Auto-generated if not provided.
detectedIntent	Not Required	Set automatically by Aza using the tool name.
finalTextMessage	Required	The text response returned by the tool. This is the primary content displayed to the user.
enhancedMessage	Optional	Rich UX elements Suggestions, SuggestionWithAction, and Links rendered in the UI. [Enhanced Message Reference]
enhancedMessage.actionItems	Optional	Array of link objects rendered as actionable buttons/links in the response.
enhancedMessage.textFormatSegments	Optional	Text formatting segments for inline styling.
imgAttributes	Optional	Array of pre-signed S3 URL strings for images to display in the response.
mfeAttributes	Optional	Array of MFE component definitions to render micro-frontend components in the response. AZA Bot-to-Bot enhanced message
sourceAttributions	Optional	Array of citation objects linked to the response content.
sourceAttributions[].citationNumber	—	Sequential citation number
sourceAttributions[].title	—	Title of the source document
sourceAttributions[].url	—	URL to the source document
sourceAttributions[].snippet	—	Relevant excerpt from the source
plugins	Not Required	Set automatically by Aza using the tool name.
systemMessageId	Optional	For tracking in Aza DB and telemetry. Auto-generated if not provided.
userMessageId	Optional	For tracking in Aza DB and telemetry. Auto-generated if not provided.

Aza supports limited HTML element rendering in responses. Supported elements include:

- Links (`<a>`)
- Buttons (`<button>`)

- Input fields (<input>)
- Submit buttons (that can submit a new chat request)

For full details on supported elements and their APIs, see the [AZA Bot-to-Bot enhanced message]

FollowUpSuggestionsEvent

The FollowUpSuggestionsEvent provides clickable suggestion chips after the response, guiding users to related queries.

Note: If your tool does not send follow-up suggestions, Aza auto-generates them.

Example:

```
{
  "type": "text",
  "text": "{\"followUpSuggestionsEvent\":{\"conversationId\":\"79c49476-e897-48d0-ac"
}
```

Structure:

Field	Required	Description
conversationId	Optional	Should match the MetadataEvent conversationId
retryOptions.shouldRetry	Optional	Whether to show a retry option
suggestions	Required	Array of suggestion objects
suggestions[].message	Required	The text displayed as a clickable suggestion chip
systemMessageId	Optional	Should match the MetadataEvent systemMessageId
userMessageId	Optional	Should match the MetadataEvent userMessageId

Combining MetadataEvent and FollowUpSuggestionsEvent

When returning a complete response, the `content` array can contain two text entries one for the MetadataEvent (mandatory) and one for the FollowUpSuggestionsEvent (optional):

```
{
  "jsonrpc": "2.0",
  "id": "req-453",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "{\"metadataEvent\": { ... }}"
      },
      {
        "type": "text",
        "text": "{\"followUpSuggestionsEvent\": { ... }}"
      }
    ],
    "isError": false
  }
}
```

Response Processing Modes

By default, Aza sends the MCP tool response back to the RBO for further LLM processing. This behavior is controlled by the `isLlmProcessingRequired` field in tool configuration.

Mode	Config Value	When to Use	How It Works
LLM Post Processing	<code>isLlmProcessingRequired:</code> true (default)	Tool returns raw/structured data (e.g., JSON) that the RBO should interpret and present as natural language	RBO receives the tool output, reasons about it, and generates a user-facing response
Response to UI Directly (Skipping RBO)	<code>isLlmProcessingRequired:</code> false	Tool returns a fully formed response with <code>MetadataEvent</code> ready to render directly in the UI	Aza bypasses RBO processing and renders the response as-is

Onboarding Steps

AUTHORIZATION

Aza only supports service-to-service auth using IAM. Aza Currently supports two types of MCP integrations.

Lambda MCP

Lambda MCPs are MCP servers in AWS Lambda that uses custom transport layer to allow MCP integration and invocation directly through lambda function URLs. [\[Amazon Guide\]](#)

To enable Lambda MCP integration

- Allow-list the following Aza service accounts in lambda resource policy:

AWS Account	Stage
770178630509	aza-services-beta
896254978035	aza-services-gamma
852072047365	aza-services-prod

- Add additional parameter to the tool config in Aza.
 - **"service": "lambda"**
 - Set endpoint as the lambda function URL.
 - Set invocation path - **"path": "/mcp"**

AgentCore Gateway/Runtime

Aza supports AgentCore Gateway and AgentCore Runtime Integrations as MCP tools.

To enable the integration:

- Allow the below AWS accounts in AgentCore resource policy

AWS Account	Stage
770178630509	aza-services-beta
896254978035	aza-services-gamma
852072047365	aza-services-prod

- Add additional parameter to the tool config in Aza.
 - **"service": "bedrock-agentcore"**
 - Add supported MCP version as **"protocol_version": ["2025-03-26"]**
 - For AgentCore Gateway
 - Set endpoint - https://{gateway_id}.gateway.bedrock-agentcore.{region}.amazonaws.com/
 - Set invocation path - **"path": "mcp"**
 - For AgentCore Runtime
 - Set endpoint - <https://bedrock-agentcore.{region}.amazonaws.com/runtimes/{url-encoded-runtime-arn}/>
 - Set invocation path - **"path": "invocations?qualifier=DEFAULT"**
- Example Config - [\[REF\]](#)

TOOL REGISTRATION IN AZA

All MCP tools are registered via the AppConfig code package [\[AZAServicesAppConfig\]](#).

1. Create a new JSON configuration file in the package
2. Always prefix Mcp to your file name (e.g., `McpDLSToolConfig.json`, `McpBMSToolConfig.json`)
3. Run the Plugin Overlap Analysis tool in local.[\[README\]](#)
4. Write formatted descriptions for easy review in CRs
5. Raise a CR the Aza team will review [\[Sample CR\]](#)
6. Once merged, configurations are polled automatically by the Aza service

Example Config

```
{
  "beta": [{
    "name": "search_documentation",
    "description": "Search AWS documentation using the official AWS Documentation",
    "example_queries": ["Can you get information about this COE with ID 5734df15-c",
    "url": "https://moyhu53grgctpovuar3hzydzya0iddsx.lambda-url.us-west-2.on.aws",
    "region": "us-west-2",
    "path": "/mcp",
    "service": "lambda",
    "inputSchema": {
      "type": "object",
      "properties": {
        "search_phrase": {
          "type": "string",
          "description": "Search phrase to use"
        },
        "limit": {
          "type": "integer",
          "minimum": 1,
          "maximum": 50,

```

```

        "default": 5,
        "description": "Maximum number of results to return"
    },
    },
    "required": ["search_phrase"]
},
"eligibilityRule": {
    "and": [
        {
            "in": [
                "POPSTAR/atoz_assistant_e360_mcp",
                {
                    "var": "features"
                }
            ]
        },
        {
            //Please define persona explicitly
            "in": [
                "HR",
                {
                    "var": "userPersona"
                }
            ]
        }
    ]
},
"hasSupportForAttachments": "false", // Aza supports file upload, but MCP serv
"isConversationHistoryRequired": "false",
"isLlmProcessingRequired": "false" //default the value is `true` if not speci
}],
"gamma": [],
"prod": []
}

```

Field	Required	Description
name	Yes	Unique name of the tool
description	Yes	LLM-optimized description used by the RBO for intent matching. Keep within 500 tokens.
example_queries	Yes	Array of sample queries. Can also be included in description using <example_queries> XML tags.
url	Yes	Host of the deployed MCP server (host only, e.g., https://example.lambda-url.com). Use custom domain for production.
region	Yes	AWS region where the server is hosted (e.g., us-west-2)
path	Yes	Path where MCP tools are accessible (e.g., /mcp)
service	Yes	"lambda" or "execute-api"
inputSchema	Yes	JSON Schema defining the tool's input parameters
eligibilityRule	Yes	JSON logic for user eligibility evaluation. Supports PAPI, BRASS, DAAPi-based attributes.
hasSupportForAttachments	Yes	"true" or "false" whether the tool accepts file attachments
isConversationHistoryRequired	Yes	"true" or "false" whether the tool needs conversation history
isLlmProcessingRequired	No	"true" (default) or "false" whether RBO should process the response before rendering

Eligibility Rules & Population Control

Eligibility rules use JSON logic and support various segmentation attributes and Popstar. Tool owners are expected to control access to their respective tools and launch dialup using the configuration.

```
{
  "eligibilityRule": {
    "and": [
      {
        "in": ["POPSTAR/atoz_assistant_e360_mcp", { "var": "features" }]
      },
      {
        "in": ["HR", { "var": "userPersona" }]
      }
    ]
  }
}
```

TELEMETRY EVENTS

To support standard metrics for all bot tenants, we need to receive all telemetry events for all of our metrics to flow in automatically. Additionally, the bot teams should also send the PXF Data Products team the bot accuracy evaluation metrics. The detailed guideline and instructions are available in the quip below.

AZA Bot to Bot Data Onboarding Guidelines

IMPORTANT:

For any platform operating independently outside AZA, the "actor" field must be explicitly populated with current actor details. Add "actor" field to telemetry payloads following this documentation:

<https://console.harmony.a2z.com/pxt-telemetry-docs/producers/reference/populating-context-on-telemetry-messages#populating-context-on-telemetry-messages>

INITIATE SECURITY/PRIVACY REVIEW

As a GenAI application, Aza requires all plugins to have ASR complete as well as Privacy compliant with Privacy review. Be aware of additional requirements including onboarding to the FAST framework and legal reviews if applicable to your use case. Start this process early to avoid launch delays.

STAGING VALIDATION QUALITY BARS (MISSING)

Quality bars for production deployment:

Routing accuracy $\geq$ 75%

Goal success rate $\geq$ 80%

Conversation score $\geq$ 80%

No regression in other plugins (verified via RBO Routing Automated Test) AZA Developer/Local Testing

Testing

LOCAL TESTING

Use the [AZA Developer/Local Testing] guide to test your MCP integration locally before submitting for review.

INTEGRATION TESTING

Each MCP server is required to set up integration tests in the Aza pipeline:

- [Tool use test cases](#)
- [Plugin test cases](#)

Tests must cover:

1. Thorough coverage of the scope of your MCP tool
2. Verification that other plugins/tools are not impacted by your addition
3. Grey areas identified and communicated to the Aza team

Check existing test results:

1. Navigate to the [AtoZAssistantServices pipeline](#)
2. Check the beta stage for "RBO Routing Automated Test"
3. Click on test results to view pass rates in the TOD portal

TESTING IN ATOZ ENVIRONMENTS

[AtoZ Beta](#) is integrated with Aza Beta. To test in AtoZ environments with population control, override your feature access:

1. Navigate to [Popstar Overrides](#)
2. Click "Create a new override"
3. Add the relevant employee ID
4. For Feature ID, add the value from `popStarFeatureIDs` relevant to your integration
5. Add a description (e.g., "Testing [INTEGRATION NAME] in [STAGE]")
6. Toggle the override to ON and create
7. Repeat with `popStarFeatureGateIDs` as the Feature ID
8. Repeat for PROD environment if needed
9. Override any other feature/feature gate IDs from federated population control
10. Allow 30 minutes for changes to propagate

FAQs

What are the expectations from MCP servers?

Every MCP server owner must register their tool with the Aza team via the AppConfig package.

What should be the expected scale for MCP server?

Aza serves 500K+ users per month. Based on your target population size, allocate sufficient capacity. Discuss scale planning with the Aza team during onboarding.

What MCP integrations and versions are supported?

Below are the MCP versions supported by Aza:

- Version `2025-06-18` - Pending development support.
- Version `2025-03-26` - Support planned by 28 Feb 2026.
- Version `2024-11-05` - Supported.

How should one host their own MCP?

An MCP server can be built using:

- [Lambda MCP](#) - MCP server running in Lambda with Lambda Function URL used for invocation.
- [AgentCore/AgentCore Gateway](#) - AWS Bedrock Agentcore with Strands agent inside or Agentcore Gateway with MCP registered over the gateway.

Can we have a common MCP server where all domain teams onboard their tools?

No. Currently, each domain team is required to build their own individual MCP server and register with the Aza application.

Are domain owners required to update Aza's tool configurations over time?

Yes. Every tool must be registered with Aza. This ensures there are no grey areas between different teams.

How should the MCP tool be modeled?

An MCP tool can be a Bedrock bot, an agent, or an integration that has multi-step workflows. The key is to model tools around user goals, not API structure.

How does conversation history filtering work if the prompt exceeds limits?

If the prompt exceeds the 200K token limit, Aza generates a contextual query or summary of the conversation history instead of sending the full history.

Does Aza support mapping multiple plugins to one MCP server?

Yes. Aza supports mapping multiple plugins to one MCP server implementation by passing the `pluginId` in the function params.

How long does it take to implement connections from the Aza side?

Approximately 3–4 days, including integration tests (FE + BE), minor code changes, and E2E testing.

Does Aza support file upload?

Yes. Aza supports file upload, but MCP servers are not required to support this feature. Inform the Aza team whether your tool plans to support file attachments.

References

- [Message preview web app \(beta\)](#)
- AZA Tenant Bot Onboarding Guide
- [MCP Hosting in Amazon Cloud](#)
- <https://w.amazon.com/bin/view/MCP/HostingInAmazonCloud/Onboarding/>
- Lambda Server - <https://code.amazon.com/packages/LambdaMCPServer/trees/mainline>
- MCP Server using OAuth - <https://code.amazon.com/packages/Guidance-for-remote-mcp-servers-on-aws/trees/main>

- Java MCP client-server integration POC - <https://code.amazon.com/packages/MetMcpPocJava/trees/heads/mcp-poc>
 - <https://modelcontextprotocol.io/introduction>
 - [AZA AppConfig \(Tool Registration\)](#)
 - [AZA Developer/Local Testing](#)
 - [AZA Onboarding Guide](#)
 - [MCP Protocol Specification](#)
 - [AWS MCP Blog](#)
-

--wiki-sync: "[A_to_Z_Wiki/AZA/AZAIIntegration/AzaMCPIntegrationGuide](#)"--